

Toronto Metropolitan University

Department of Electrical, Computer, & Biomedical Engineering

Faculty of Engineering & Architectural Science

Course Title:	Distributed Cloud Computing
Course Number:	COE892
Semester/Year:	W2026
Instructor:	Dr. Muhammad Jaseemuddin
Assignment/Lab Number:	Project
Assignment/Lab Title:	Project Proposal
Submission Date:	6 February 2026
Due Date:	6 February 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Khan	Muhammad	501055504	8	M.K
Hassan	Shaheer	501053803	8	S.H
Arif	Abdullah	501122593	8	A.A

**By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct.*

Distributed Air Traffic Control System

The Problem

Air traffic control (ATC) is one of the most safety-critical distributed systems in the real world. Controllers must simultaneously monitor dozens of aircraft across multiple airspace sectors, coordinate takeoffs and landings on shared runways, manage ground taxiing, and respond to emergencies—all in real time with zero tolerance for error. A failure or delay in any single component can cascade into dangerous conflicts or collisions. Traditional ATC systems are centralized and monolithic, creating single points of failure and making it difficult to scale as air traffic increases. This project aims to design and implement a distributed air traffic control simulation that addresses these challenges by decomposing the system into independent, communicating microservices—directly applying the principles of distributed cloud computing studied in COE892.

The Solution

Our distributed ATC system will simulate a multi-airport environment where aircraft move through controlled airspace sectors. The system is decomposed into the following cooperating services:

1. **Radar Simulation Service:** Generates and broadcasts real-time aircraft position updates (latitude, longitude, altitude, speed, heading) at configurable intervals, simulating radar sweeps across the airspace.
2. **Sector Control Services:** Multiple independent instances, each responsible for a defined airspace sector. Each service monitors aircraft within its sector, detects potential conflicts (separation violations), and issues altitude/heading advisories. Aircraft are handed off between sectors via inter-service messaging.
3. **Runway & Ground Control Service:** Manages runway allocation for takeoffs and landings using a distributed queue. Enforces sequencing rules (e.g., minimum separation time between operations) and handles taxiway routing to prevent ground conflicts.
4. **Conflict Detection & Resolution Service:** A centralized safety layer that cross-checks all sector data for potential mid-air conflicts and issues resolution commands (altitude changes, holding patterns) when separation minimums are violated.
5. **Real-Time Dashboard:** A web-based frontend that visualizes live aircraft positions on a map, displays active sector boundaries, runway status, conflict alerts, and system health metrics via WebSocket streaming.

All services communicate asynchronously through a message broker, ensuring loose coupling and fault tolerance. Each service is containerized with Docker and orchestrated using Docker Compose, allowing the entire multi-node system to be deployed and scaled independently.

Key Technologies

- **Python (FastAPI):** High-performance async framework for building the backend microservices and REST APIs.

- **RabbitMQ:** Message broker for asynchronous inter-service communication (aircraft position broadcasts, handoff messages, conflict alerts).
- **WebSockets:** Real-time bidirectional communication for streaming live aircraft positions and alerts to the dashboard.
- **Docker & Docker Compose:** Containerization of each microservice for consistent deployment and independent scaling of sector controllers.
- **Redis:** In-memory data store for shared state (current aircraft positions, runway queues) with low-latency access across services.
- **React.js with Leaflet:** Interactive map-based dashboard for visualizing aircraft, sectors, runways, and real-time conflict alerts.
- **PostgreSQL:** Persistent storage for flight plans, historical logs, and system audit data.

Proposed Timeline

Week	Deliverables
Week 6	Set up project repository, Docker environment, and CI pipeline. Define service interfaces and message schemas. Implement the Radar Simulation Service with configurable aircraft generation.
Week 7	Implement Sector Control Services with aircraft tracking, sector boundary logic, and inter-sector handoff via RabbitMQ. Deploy multiple sector instances.
Week 8	Build Runway & Ground Control Service with distributed queue-based sequencing. Implement the Conflict Detection & Resolution Service with separation monitoring.
Week 9	Develop the React/Leaflet real-time dashboard with WebSocket integration. Connect all services end-to-end and validate data flow.
Week 10	Fault tolerance testing: simulate node failures, message loss, and network partitions. Performance tuning and load testing with increased aircraft count.
Week 11	Final integration testing, bug fixes, documentation, and demo preparation. Submit final report and codebase.

Team Member Responsibilities

****Team responsibilities are subject to change as the project progresses, if required.****

Member(s)	Responsibilities
Muhammad Khan	• Design and implement the Radar Simulation Service and aircraft data models • Set up Docker containerization and Docker Compose orchestration for all services

	• Configure RabbitMQ message broker, topic exchanges, and message schemas • Conduct fault tolerance testing (node failures, message loss, network partitions)
Shaheer Hassan	• Implement Sector Control Services with aircraft tracking and sector boundary logic • Develop inter-sector handoff protocol and separation monitoring algorithms • Build the Conflict Detection & Resolution Service with safety-critical alerting • Set up Redis for shared state and PostgreSQL for persistence and audit logging
Abdullah Arif	• Build Runway & Ground Control Service with queue-based takeoff/landing scheduling • Develop React/Leaflet real-time dashboard with WebSocket streaming • Implement conflict alert display, runway status panels, and system health metrics • Write project documentation, README, final report, and demo preparation